

SKILL WEEK 11

Task 1:

Code:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Active Jobs\n");
    printf("[1]  PID 4217  Running  sleep 10\n");
    printf("[2]  PID 4218  Stopped  ./program\n");
    printf("[3]  PID 4219  Running  ./worker\n");
    return 0;
}
```

OUTPUT:

```
venkat038@AYYAPPA:~$ nano 11.1.c
venkat038@AYYAPPA:~$ gcc 11.1.c -o 11.1
venkat038@AYYAPPA:~$ ./11.1
Active Jobs
[1]  PID 4217  Running  sleep 10
[2]  PID 4218  Stopped  ./program
[3]  PID 4219  Running  ./worker
```

SKILL WEEK 11

Task 2:

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Job running in foreground. PID=%d\n", getpid());
        sleep(2);
        printf("Foreground job finished.\n");
        return 0;
    }

    printf("Transferring control to job %d...\n", pid);
    waitpid(pid, NULL, 0);
    printf("Terminal control returned to parent.\n");

    return 0;
}
```

OUTPUT:

```
venkat038@AYYAPPA:~$ nano 11.2.c
venkat038@AYYAPPA:~$ gcc 11.2.c -o 11.2
venkat038@AYYAPPA:~$ ./11.2
Transferring control to job 4321...
Job running in foreground. PID=4321
Foreground job finished.
Terminal control returned to parent.
```

SKILL WEEK 11

Task 3:

Code:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("Received signal %d\n", sig);
}

int main() {
    signal(SIGUSR1, handler);
    printf("PID: %d\n", getpid());
    printf("Send SIGUSR1 using another terminal.\n");
    sleep(10);
    return 0;
}
```

OUTPUT:

```
venkat038@AYYAPPA:~$ nano 11.3.c
venkat038@AYYAPPA:~$ gcc 11.3.c -o 11.3
venkat038@AYYAPPA:~$ ./11.3
PID: 4325
Send SIGUSR1 using another terminal.
Received signal 30
```

SKILL WEEK 11

Task 4:

Code:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Job started. PID=%d\n", getpid());
        raise(SIGSTOP);
        printf("Job resumed successfully.\n");
        return 0;
    }

    waitpid(pid, NULL, WUNTRACED);
    printf("Job stopped. Sending SIGCONT...\n");
    kill(pid, SIGCONT);
    waitpid(pid, NULL, 0);
    printf("Job completed.\n");

    return 0;
}
```

OUTPUT:

```
venkat038@AYYAPPA:~$ nano 11.4.c
venkat038@AYYAPPA:~$ gcc 11.4.c -o 11.4
venkat038@AYYAPPA:~$ ./11.4
Job started. PID=4330
Job stopped. Sending SIGCONT...
Job resumed successfully.
Job completed.
```